

TP1

1.a

- Mencione y explique las características más relevantes de GNU/Linux.
- GNU/Linux es un sistema operativo libre que surge de la combinación del núcleo Linux con el sistema GNU.

Está compuesto principalmente por un kernel, un sistema de archivos jerárquico y un intérprete de comandos. El kernel es monolítico modular. Es decir se comporta como un único programa en memoria, pero tiene capacidad de cargar y descargar módulos mientras se ejecuta. El kernel se distribuye bajo la licencia libre GPLv2.

Es un sistema operativo multiusuario y multitarea escrito principalmente en C y soporta varias arquitecturas.

El sistema de archivos provee un modelo de permisos basado en tres categorías, el usuario, su grupo y los demás.

1.b

- Mencione otros sistemas operativos y compárelos con GNU/Linux en cuanto a los puntos mencionados en el inciso a.
- Windows es un sistema operativo comercial privativo de código cerrado. Además, Windows es desarrollado principalmente por Microsoft, tanto el kernel como sus utilidades de usuario. El kernel de Windows tiene incorporado más elementos que en Linux son programas de usuario. Pero es también híbrido. Windows esta escrito en C principalmente pero al ser de código cerrado su portabilidad es más limitada en la practica. El sistema de archivos que provee Windows es muy diferente al modelo de Linux en estructura y funcionalidad. Windows soporta FAT, exFAT, y NTFS pero Linux tiene una capa de abstracción, VFS, que permite incorporar nuevos y soporta bastantes.
- Android esta basado en el kernel Linux y es desarrollado por Google y otros, si bien es de código abierto, esta optimizado para las pantallas táctiles y dispositivos ARM. Esto limita su portabilidad en la practica si ademas tenemos en cuenta que sus utilidades de usuario son limitadas. Android usa las utilidades de usuario **Toybox**, entre otras, que se caracterizan por ser bastante mínimas en comparación a sistemas GNU.

1.c

- ¿Qué es GNU?

- GNU es un sistema operativo tipo Unix. desarrollado para el proyecto GNU y bajo el auspicio de la FSF. Formado principalmente de software libre bajo términos *copyleft* como GPL. GNU significa *GNU is not Unix*, que destaca la diferencia entre GNU y Unix, siendo el primero libre y el segundo privativo. El kernel del proyecto, GNU Hurd, no llegó a completarse. Algunos componentes populares son **Bash**, **coreutils**, **gcc** y **glibc**.

1.d

- Indique una breve historia sobre la evolución del proyecto GNU.
- Richard Stallman inicia el proyecto en 1983. En 1985 crea la FSF que proporcionará soporte técnico y legal al proyecto GNU. Para el 1990 se han creado GCC, Bash e Emacs y la mayor parte de bibliotecas y utilidades que conformarían el SO, pero no se tiene un kernel funcional. Durante los 80 el proyecto GNU evalúa, e intenta adaptar kernels existentes. A principios de los 90 se opta por adaptar Mach de la Universidad Carnegie Mellon, que se denominaría Hurd. Sin embargo, este se demoraría y su adopción es limitada. De forma autónoma empieza el desarrollo de un kernel por el informático finlandés Linus Torvalds en 1991. Torvalds adoptaría la licencia GPL para su proyecto en el año 1992. Esto precipita el desarrollo del kernel que tomaría el nombre Linux, y que da lugar a la adopción de este junto a las herramientas GNU integradas en un SO libre que predomina hasta hoy.

1.e

- Explique qué es la multitarea, e indique si GNU/Linux hace uso de ella.
- La multitarea es la capacidad de un SO de conmutar y administrar múltiples tareas, es decir, procesos e hilos, de forma concurrente. GNU/Linux es multitarea. Específicamente usa multitarea apropiativa: el kernel puede interrumpir una tarea en ejecución para asignar tiempo de CPU a otra.

1.f

- ¿Qué es POSIX?
- POSIX es una norma de la IEEE que define una interfaz estándar del SO y su entorno, incluyendo al intérprete de comandos y programas de utilidades con el objetivo de soportar la portabilidad de aplicaciones, esta inspirada en interfaces, intérpretes de comandos y utilidades comunes en versiones de UNIX. Hoy en día muchos SO cumplen parcial o totalmente con algún estándar POSIX.

2.a

- ¿Qué es una distribución de GNU/Linux? Nombre al menos 4 distribuciones de GNU/Linux y cite diferencias básicas entre ellas.

- Una distribución de GNU/Linux es un gran conjunto de programas binarios empaquetados para ser distribuidos. En una distribución generalmente se incluyen programas de instalación que facilitan el proceso de configuración e instalación del SO.

Algunas distribuciones son Debian, Ubuntu, Fedora, Arch. las principales diferencias son sus administradores de paquetes, los mecanismos de actualización y las metodologías de desarrollo.

2.b

- ¿En qué se diferencia una distribución de otra?
- Generalmente usan gestores de paquetes diferentes. Tienen un modelo de lanzamientos diferente, es diferente el propósito que cumplen, el método de instalación, o su filosofía.

2.c

- ¿Qué es Debian? Acceda al sitio <https://www.debian.org/> e indique cuáles son los objetivos del proyecto y una breve cronología del mismo.
- Debian es una distribución GNU/Linux desarrollada por el proyecto Debian. Entre sus objetivos se encuentran: el crear un sistema operativo completamente libre y accesible a todos, mantener el software totalmente libre, ser estable, seguro y de alta calidad, ser universal y servir de base para otras distribuciones y contribuir de vuelta a la comunidad de software libre.

Históricamente Debian, fundado en 1993 por Ian Murdock tiene su primera versión 1.1 llamada Debian Buzz en el año 1996. Actualmente nos encontramos en su versión 12 Debian Bookworm que fue lanzada en 2023.

3.a

- Nombre cuáles son los componentes fundamentales de GNU/Linux.
- Un SO GNU/Linux está compuesto principalmente por el kernel Linux, una biblioteca C estándar como glibc, utilidades de usuario GNU, un intérprete de comandos como Bash, un gestor de arranque como GRUB, un sistema de inicio como systemd, un sistema de archivos, y otros componentes de bajo nivel como PulseAudio, Mesa, etc. según su necesidad.

3.b

- Mencione y explique la estructura básica del Sistema Operativo GNU/Linux.
- Un SO GNU/Linux tiene una estructura en capas. En la capa más interna está el kernel, que se ejecuta en modo privilegiado. Por encima de este están los manejadores, que a veces son parte del kernel. Encima está la biblioteca C estándar. Esta última está en el espacio de usuario junto con las demás aplicaciones de usuario como las utilidades GNU, bash y demás comandos.

4.a

- ¿Cuáles son las funciones principales del kernel?
- Las funciones principales del kernel son:
 - Proveer a los usuarios de abstracciones útiles por medio de las llamadas al sistema.
 - Administrar los recursos de hardware de forma equitativa, eficiente y segura según se requiera.
 - Gestionar peticiones de entrada/salida de los procesos.

4.b

- ¿Es posible tener más de un kernel de GNU/Linux instalado en la misma máquina? (ayuda /boot).
- Si es posible, como ejemplo en una distribución Ubuntu en el directorio /boot se puede tener: `vmlinuz`, `vmlinuz-7.0.0-29-generic`, `vmlinuz-7.0.0-30-generic`, `vmlinuz.old`. Todas son imágenes del kernel.

4.c

- ¿En qué directorio se encuentra ubicado el kernel?
- Como se insinuó está en /boot. Estas entradas son las que es posible seleccionar desde el menú que presenta normalmente GRUB.

5.a

- ¿Qué es y cuáles son las funciones del intérprete de comandos o *shell*?
- El intérprete de comandos es un programa que presenta una interfaz al usuario para poder interactuar con el SO. Una shell permite ejecutar programas en el SO, como comandos y aplicaciones. Además Bash, por ejemplo, soporta redireccionamiento, tuberías, alias, historial, etc.

5.b

- Investigue y mencione al menos 3 intérpretes de comandos que existen para GNU/Linux y compárelos entre ellos.
- Tres shells son csh, zsh y bash. Hacer una comparacion relevante de diferencias me es imposible en este momento.:(

5.c

- ¿Dónde se ubican (path) los comandos propios y externos al Shell?
- Los comandos propios de la shell son parte de esta y se ejecutan internamente, por ej. `cd`, alias, etc.
En cambio los externos se ubican en archivos que la shell busca en los directorios de la v. de entorno `PATH`, típicamente en `/bin`, `/usr/bin`. Entre otros.

5.d

- ¿Por qué considera que la Shell no es parte del Kernel de GNU/Linux?
- La shell no forma parte del kernel por razones de diseño. No existe necesidad de que esta forme parte del kernel.

Para empezar, la shell no interactúa con hardware, puede usar `glibc` y ejecutar sus comandos, etc. Es decir, la shell es un programa de usuario que aprovecha las abstracciones del kernel y otras bibliotecas, por esto, no es necesario que forme parte de este.

5.e

- ¿Es posible definir una distinta para cada usuario?
- Si, cada usuario tiene definida una shell de inicio que se guarda en el archivo `/etc/passwd`.

6.a

- ¿Qué es un sistema de archivos en Linux?
- Un sistema de archivos es la estructura y el conjunto de reglas que utiliza el sistema operativo para organizar, almacenar y recuperar datos en un dispositivo de almacenamiento.

Caracterizado principalmente por presentar una jerarquía de árbol único que parte del directorio raíz, donde todos los dispositivos y particiones se montan como sub-directorios.

6.b

- ¿Cuál es la estructura básica de los File System en GNU/Linux? Mencione los directorios más importantes e indique qué tipo de información se encuentra en ellos. ¿A qué hace referencia la sigla FHS?
- La estructura básica de un sistema de archivos en GNU/Linux es la de árbol. Con su raíz en `/`.

Sus directorios más importantes son:

1. `/bin`: a donde se almacenan los binarios de comandos accesibles a todos los usuarios.
2. `/boot`: archivos estáticos que carga el gestor de arranque.
3. `/dev`: archivos de dispositivos.
4. `/etc`: configuración del sistema.
5. `/home`: directorios de archivos de los usuarios.
6. `/lib`: librerías compartidas y módulos del kernel.
7. `/media`: donde se montan medios extraíbles.
8. `/mnt`: donde se montan temporalmente sistemas de archivos.

9. **/proc**: Sistema de archivos virtual que contiene información de los procesos en ejecución y del kernel en tiempo real
10. **/root**: home del superusuario.
11. **/run**: datos de variables en tiempo de ejecución.
12. **/sbin**: archivos binarios del sistema.
13. **/srv**: archivos de servicios provistos por el sistema.
14. **/tmp**: archivos temporales.
15. **/usr**: Almacena la mayoría de los programas, librerías, documentación y archivos de código fuente del sistema para los usuarios.
16. **/var**: logs, etc.

FHS, hace referencia a *Filesystem Hierarchy Standard*, es un estándar que regula la ubicación de archivos y directorios en SO tipo Unix.

6.c

- Mencione sistemas de archivos soportados por GNU/Linux
- Los principales sistemas de archivos modernos son **ext4**, **btrfs**, y **XFS**.

7.a

- Particiones: definición, tipos, ventajas y desventajas.

Las particiones son divisiones lógicas de un medio de almacenamiento. El sistema operativo las trata como dispositivos independientes.

Existen dos esquemas principales de particionado: **MBR** y **GPT**.

- **MBR**: soporta hasta 4 particiones primarias; para obtener más particiones se crea una partición extendida que contiene particiones lógicas. Está limitado a discos de hasta 2 TB.
- **GPT**: no tiene el límite práctico de 4 particiones primarias, soporta discos de gran tamaño, incluye una copia de seguridad de la tabla de particiones y utiliza CRC para detectar corrupción.

Ventajas del particionado:

- Aislamiento de fallos: si una partición se corrompe, las demás pueden permanecer intactas.
- Posibilidad de instalar múltiples sistemas operativos.
- Uso de diferentes sistemas de archivos según los requerimientos de cada partición.

Desventajas del particionado:

- Poca flexibilidad por tener un tamaño fijo.
- Posible desperdicio de espacio libre si las particiones no se dimensionan adecuadamente.
- Redimensionar particiones puede implicar riesgos o requerir herramientas específicas.

7.b

- ¿Cómo se identifican las particiones en GNU/Linux? (Considere discos IDE, SCSI y SATA).
- las particiones se identifican mediante archivos especiales ubicados en `/dev`. En el caso de IDE `hda`, `hdb`, etc. corresponden a discos y `hda1`, `hda2` se corresponden a particiones de `hda`. En el caso de SATA y SCSI se usa `sda`, `sdb`, etc. para discos y `sda1`, `sda2` para particiones.

7.c

- ¿Cuántas particiones son necesarias como mínimo para instalar GNU/Linux? Nómbrelas indicando tipo de partición, identificación, tipo de File System y punto de montaje.
- Técnicamente la única partición imprescindible es la de la raíz. Que podría tener la identificación `sda2` y puede tener una variedad de sistemas de archivos, su punto de montaje es `/`. En sistemas UEFI se necesita una partición EFI. Que debe ser FAT32 y tiene punto de montaje `/boot/efi`. Opcionalmente se usa también una partición de intercambio.

7.d

- Dar ejemplos de diversos casos de particionamiento dependiendo del tipo de tarea que se deba realizar en su sistema operativo.
- En una estación de trabajo bastaría con separar `/home` y tener las particiones necesarias `/` y `/boot/efi`. Opcionalmente podemos tener swap.
En un servidor web podríamos separar `/var` y `/tmp`, además de separar `/srv` o `/home` para aislar logs, temporales y datos del servicio de la raíz.
En un sistema embebido usualmente hay una raíz de solo lectura y algunas particiones de escritura para configuración o datos de aplicación.

7.e

- ¿Es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en GNU/Linux?
- Si, no solo es posible la lectura si no también la escritura. Además en el caso de FAT además este es el sistema de archivos obligatorio para la partición EFI. Por lo que la mayoría de distribuciones tienen una partición FAT.

7.f

- ¿Qué tipo de software para particionar existe? Menciónelos y compare.
- Principalmente están `fdisk` y `gparted`. Por un lado, `fdisk` es una de interfaz de texto que no formatea, no permite re dimensionamiento, pero, está disponible por defecto en la mayoría de distribuciones. Por otro lado, `gparted` es una utilidad gráfica que si hace formateo y re dimensiona particiones pero puede requerir instalación y un entorno gráfico.

8.a

- ¿Qué es el BIOS? ¿Qué tarea realiza?
- el BIOS de *Basic I/O System*, es un programa que se ejecuta al arrancar una computadora x86 y se encarga principalmente de inicializar el hardware y cargar un gestor de arranque para que el SO tome el control.
Primero realiza un POST, *power on self test* que verifica cuales dispositivos tiene disponibles la computadora y su funcionamiento.
Configura los dispositivos básicos según la configuración almacenada en su memoria.
Busca un dispositivo de almacenamiento que tenga un sector de arranque válido según su configuración
lee el primer sector físico de disco y carga el gestor de arranque transfiriéndole el control.
Durante el arranque y incluso después ofrece rutinas para E/S, que permiten interactuar con el teclado, pantalla y discos.

8.b

- ¿Qué es UEFI? ¿Cuál es su función?
- UEFI de *Unified Extensible Firmware Interface* es un estándar que regula la interfaz entre el firmware de la plataforma, hardware, y el SO durante el arranque.
Su función principal es la de inicializar el hardware y cargar el SO ofreciendo un entorno estándar que permita la comunicación entre el SO y el firmware empujado.
Por este motivo realiza las tareas:
 - Un auto-diagnóstico parecido al de BIOS.
 - Cargar los controladores de los dispositivos básicos.
 - Leer la tabla de particiones GPT
 - Localizar y ejecutar el cargador de arranque, archivo .efi, dentro de la partición ESP.
 - Ofrecer un entorno de pre-arranque, un intérprete de comandos.

Además, UEFI proporciona servicios de tiempo de ejecución, que permiten al sistema operativo seguir comunicándose con el firmware incluso después de haber tomado el control. Por ejemplo UEFI brinda la infraestructura necesaria para la interfaz ACPI, que permite al sistema operativo gestionar el consumo energético de los dispositivos y controlar los estados de suspensión del sistema.

8.c

- ¿Qué es el MBR? ¿Qué es el MBC?
- MBR de *master boot record* es el primer sector físico de un medio de almacenamiento, encargado de cargar el SO y definir cómo está particionado el disco. Este sector de 512 bytes tiene tres partes:
 1. MBC de *master boot code*, ocupa los primeros 446 bytes y es el código que la BIOS carga en memoria y ejecuta al arrancar. Su función es: analizar la tabla de particiones, buscar la que es activa, localizar el sector de inicio de la partición activa, cargarlo en memoria y, finalmente, transferir el control a este código que ya es específico del SO.
 2. Tabla de particiones, ocupa 64 bytes, describe cómo está particionado el disco, tiene un máximo de 4 entradas y por ende ese límite de particiones primarias.
 3. Firma de arranque, que ocupa 2 bytes y sirve como indicador de que este disco es válido para arrancar.

8.d

- ¿A qué hacen referencia las siglas GPT? ¿Qué sustituye? Indique cuál es su formato.
- Hacen referencia a *GUID Partition Table*. Un esquema de partición que reemplaza a MBR. En especial en sistemas con arranque EFI.
Su formato incluye:
 1. MBR de protección: mantiene compatibilidad.
 2. Cabecera GPT primaria: contiene la firma 'EFI PART', el tamaño de la tabla de particiones, el GUID del disco, la cantidad de entradas y un checksum.
 3. Entradas de partición: cada una describe una partición con GUID único, tipo, LBA de inicio y fin y sus atributos.
 4. Cabecera GPT de respaldo

8.e

- ¿Cuál es la funcionalidad de un Gestor de Arranque? ¿Qué tipos existen? ¿Dónde se instalan? Cite gestores de arranque conocidos.
- Un gestor de arranque tiene la tarea de localizar el SO en el disco, cargarlo en memoria y cederle el control. Entre sus funciones están:
 1. Cargar el kernel, y una imagen de *disco RAM*
 2. Pasar parámetros al kernel.
 3. Cargar y ejecutar aplicaciones *.efi*.
 4. Presentar un menú con opciones de SO y versiones del kernel.

En sistemas BIOS el gestor se instala en el MBR, mientras que en UEFI se instala en la partición EFI, `/boot/efi`.

Existen varios tipos, avanzados como GRUB 2, o simples como LILO y Windows Boot Manager.

8.f

- ¿Cuáles son los pasos que se suceden desde que se prende una computadora hasta que el SO es cargado (proceso de *bootstrap*)?
- la computadora ejecuta la BIOS/UEFI.
- realiza un auto diagnostico, y carga controladores básicos y carga el gestor de arranque ya sea desde el MBR o la partición EFI.
- el gestor de arranque carga el kernel del SO, y a menudo un *initrd/initramfs*.
- el kernel se inicializa y monta la raíz.
- se ejecuta el primer proceso, *init/systemd*.

8.g

- Analice el proceso de arranque en GNU/Linux y describa sus principales pasos.
 1. Encendido y ejecución de firmware.
 2. Auto diagnóstico e inicialización de hardware: se verifica el estado de memoria, discos, etc, se inicializan los controladores básicos de dispositivo y se define el orden de arranque.
 3. Localización y carga del gestor de arranque.
 4. Ejecución del gestor de arranque: GRUB carga en memoria el kernel `vmlinuz-`. Carga un disco RAM `initrd.img-`. Pasa parámetros al kernel, `root=/dev/sda1, ro, quiet`.
 5. Inicialización del kernel: se descomprime el kernel y empieza su inicialización, configura la gestión de memoria, detecta e inicializa dispositivos de hardware y carga el sistema de archivos raíz temporal mediante `initramfs`. Una vez cargados los controladores esenciales, se monta el sistema de archivos real y se realiza el cambio de raíz mediante `switch_root`.
`initramfs` contiene los drivers y herramientas esenciales que permiten montar el sistema de archivos real.
 6. Ejecución del primer proceso: se ejecuta el primer proceso de usuario, `init` o `systemd`. Tiene PID 1 y es el responsable de iniciar todos los procesos del sistema.
 7. Arranque de servicios y entorno de usuario: el sistema de inicio lanza servicios de red, cron, syslog, etc., terminales virtuales y si corresponde, el entorno gráfico. El sistema está listo entonces para iniciar sesión.

8.h

- ¿Cuáles son los pasos que se suceden en el proceso de parada (*shutdown*) de GNU/Linux?
 1. Inicio del apagado: el usuario ejecuta un comando de apagado, `shutdown -h now` o `systemctl poweroff` y el sistema de inicio recibe la orden.
 2. Notificación a los usuarios: un mensaje vía `wall` que advierte a los usuarios.
 3. Detención ordenada de los servicios: se ejecutan los scripts de parada en orden inverso que fueron iniciados. En `systemd` se detienen las unidades activas.
 4. Terminación de procesos de usuario: se terminan los procesos de los usuarios, cerrando sus sesiones y aplicaciones.
 5. Sincronización de datos: se ejecuta el comando `sync` para escribir en disco los datos pendientes de la caché.
 6. Desmontaje de sistemas de archivos: se desmontan todos, excepto el raíz, que se remonta como solo lectura.
 7. Detención de procesos restantes: se envía `SIGTERM` y si no responde, `SIGKILL`.
 8. Apagado de hardware: el kernel ejecuta rutinas de apagado de hardware como discos, puertos, etc. Finalmente el sistema se detiene, `halt` o `poweroff`.

8.h

- ¿Es posible tener en una PC GNU/Linux y otro SO instalado? Justifique. Si, cada SO puede instalarse en su propia partición o disco. En sistemas modernos con UEFI cada SO tiene su gestor de arranque en la partición EFI. Y el gestor de arranque, por ej. GRUB presenta un menú con las opciones disponibles.

9.a

- ¿Cómo se identifican los archivos en GNU/Linux? En GNU/Linux no existe el concepto de extensión de archivo. Un archivo se identifica por su contenido, tipo, es decir directorio, enlace, etc, y sus permisos. Herramientas como `file` permiten determinar el tipo.

9.e

- `ln -s /testing /testOnRoot`, crea un enlace simbólico al archivo `/testing` en `/`.
- `cp /testing /testingCopy`, crea una copia del archivo `/testing` en el mismo directorio.
- `find [ruta/s..] [expresion]`

```
find /home -name "TP1.pdf"
find /home -iname "*.JPG"
```

11

- investigue el objetivo, parámetros y ubicación de los siguientes comandos:
- **man**: una interfaz para los manuales del sistema. Tiene la sintaxis básica **man [seccion] palabra**, las secciones más comunes son del 1 al 9, y se provee el nombre de alguna utilidad, llamada al sistema, libc, etc. Por ej. **man 2 write** para llamadas al sistema. Usa un pager, normalmente **less**, para mostrar las páginas.

Parámetros útiles:

- -f: da una descripción breve.
- -w: da la ruta de la pagina del manual.
- -a: muestra todas las coincidencias en todas las secciones del manual en sucesión.
- -k: busca en nombre o descripción.

Las páginas están guardadas en **/usr/share/man/** y el ejecutable en **/usr/bin/man**.

- **shutdown**: usado para apagar, reiniciar o *halt*. Tiene los parámetros: -h para apagar, -r para reiniciar, -H para halt. Tiene la sintaxis **shutdown -h now** o **shutdown -r +5** el ultimo comando reinicia el sistema en cinco minutos. Se ubica en **/usr/sbin/shutdown**.
- **reboot**: reinicia el sistema, algunos parámetros son -f para forzar el reinicio evitando systemd. y -p para apagar el sistema. Su ubicación es en **/usr/sbin/reboot**.
- **halt**: detiene los servicios, desmonta los sistemas de archivos y a veces no apaga la máquina. Se ubica en **/usr/sbin/halt**.
- **uname**: imprime información del sistema, algunos parámetros importantes son -a para imprimir todo, -m para imprimir la arquitectura, -r para imprimir la versión del kernel. Se ubica en **/usr/bin/uname**.
- **dmesg**: muestra los mensajes generados durante el arranque y por controladores de dispositivos. Algunos parámetros son -w para esperar mensajes, -T para ver el tiempo de forma legible. Su ubicación es **/usr/bin/dmesg**.
Por ejemplo: **dmesg -T | tail, dmesg | grep usb, dmesg -l err**.
- **lspci**: muestra información de los buses PCI/PCIE y dispositivos conectados. Algunos parámetros son -v para mostrar detalles y -k para mostrar el controlador de cada dispositivo. Su ubicación es **/usr/bin/lspci**. lspci forma parte de pciutils.
- **at**: ejecuta un comando en un momento especificado. Ej **at -f /ruta/al/script.sh 14:00**. Ubicación en **/usr/bin/at**.
- **head**: muestra las primeras lineas de un archivo. Algunos parámetros -n para mostrar n lineas. Ubicación **/usr/bin/head**.
- **tail**: **/usr/bin/tail**.

12.a

- **Proceso de arranque System V:** enumere los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.
 1. Se ejecuta el código del BIOS.
 2. El BIOS ejecuta el POST (autodiagnóstico).
 3. El BIOS lee el sector de arranque (MBR).
 4. Se carga el gestor de arranque (por ejemplo, GRUB).
 5. El gestor de arranque carga el kernel y el initrd/initramfs.
 6. El kernel se descomprime, inicializa el scheduler y el resto del hardware; monta el initrd/initramfs como raíz temporal, carga los controladores necesarios y luego monta el sistema de archivos raíz real.
 7. El kernel ejecuta `/sbin/init` (el primer proceso) y se libera el initrd.
 8. `init` lee `/etc/inittab` para determinar el runlevel por defecto.
 9. Se ejecutan los scripts de inicialización del sistema (runlevel S o 1).
 10. Se cambia al runlevel por defecto y se ejecutan los scripts correspondientes.
 11. Se inician los servicios y los procesos de login (`getty`).
 12. El sistema está listo para que el usuario inicie sesión.

12.b

- Proceso de arranque System V: el proceso INIT. ¿Quién lo ejecuta? ¿Cuál es su objetivo?

INIT es ejecutado por el kernel luego de inicializar el hardware y la gestión de memoria, montar el sistema de archivos raíz real y liberar initramfs.

Su objetivo es:

- Inicializar el sistema de acuerdo al *runlevel* seleccionado.
- Lee el archivo `/etc/inittab` que define: runlevel por defecto, los scripts de arranque para cada runlevel, las terminales virtuales para el inicio de sesión.
- Gestionar los runlevels: cambiar de un nivel de ejecución a otro, por ej. de monousuario a multiusuario.
- Arrancar y detener servicios: usando los scripts en `/etc/init.d`.
- Adoptar procesos huérfanos.
- Proporcionar el entorno final: para que el usuario pueda iniciar sesión.

12.c

- Proceso de arranque System V: runlevels. ¿Qué son? ¿Cuál es su objetivo?

Los *runlevels* o niveles de ejecución son estados predefinidos del sistema en SysV Init. Que determinan que conjunto de servicios y procesos deben estar activos en un momento dado.

Cada *runlevel* esta asociado a un numero y define:

- Que servicios se inician o detienen.
- Si el sistema está en modo monousuario o multiusuario.
- Si hay interfaz gráfica o solo consola.
- Si se está apagando o reiniciando.

12.d

- Proceso de arranque System V: ¿A qué hace referencia cada nivel de ejecución según el estándar? ¿Dónde se define qué *runlevel* ejecutar al iniciar el sistema operativo? ¿Todas las distribuciones respetan estos estándares?

En SysV init los runlevels estándar son:

- 0 *halt*, apagar el sistema.
- 1 1/S modo monousuario, solamente root, mantenimiento.
- 2 multiusuario sin servicios de red.
- 3 multiusuario con red.
- 4 no estándar, disponible para usos específicos.
- 5 multiusuario con entorno gráfico.
- 6 *reboot*, reiniciar el sistema.

El *runlevel* que se ejecuta al iniciar el sistema se define en */etc/inittab*.

No todas las distribuciones respetan esta convención, y el comportamiento en sistemas modernos es emulado mediante *targets* de systemd. Por ej. *poweroff.target*, *graphical.target*, *reboot.target*.

12.e

- Proceso de arranque System V: Archivo */etc/inittab*. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en él? ¿Cuál es la estructura de la información que en él se almacena?

/etc/inittab es el archivo de configuración principal del proceso *init*. Su finalidad es definir el comportamiento durante el arranque y la operación del sistema. */etc/inittab* contiene lineas de configuración que especifican:

- *initdefault*, el runlevel por defecto.
- *sysinit*, scripts o programas que se ejecutan al iniciar el sistema.

- procesos que se lanzan en un runlevel determinado.
- terminales virtuales para login.

Cada linea de *inittab* tiene el formato:

id:runlevels:accion:proceso

Un ejemplo típico seria

```
# Runlevel por defecto
id:3:initdefault:

# Inicializacion del sistema
si::sysinit:/etc/rc.d/rc.sysinit

# Ejecutar scripts del runlevel 3
l3:3:wait:/etc/rc.d/rc 3

# Terminales virtuales
1:2345:respawn:/sbin/getty 38400 tty1
2:2345:respawn:/sbin/getty 38400 tty2

# Accion de Ctrl+Alt+Del
ca:12345:ctrlaltdel:/sbin/shutdown -r now
```

12.f

- Proceso de arranque System V: Suponga que se encuentra en el runlevel <X>. Indique qué comando(s) deberá ejecutar para cambiar al runlevel <Y>. ¿Este cambio es permanente? ¿Por qué?

Para cambiar al runlevel <Y> bastaría con usar el comando **sudo telinit Y**. El cambio es temporal, y solo modifica el nivel de ejecución actual; no la configuración de arranque (*inittab*).

12.g

- Proceso de arranque System V: scripts RC. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene se ejecutan scripts, indique cómo determina qué script ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justifique.

Los *scripts RC* de *run control* son scripts de inicio y parada de servicios. Su objetivo es iniciar, detener y reiniciar servicios del sistema de forma ordenada de acuerdo al runlevel activo. Su finalidad es automatizar la configuración del entorno como red, servicios y demonios.

Se almacenan principalmente en **/etc/init.d** por ejemplo **/etc/init.d/ssh**. Y también están los enlaces simbólicos **/etc/rcX.d** por ejemplo **/etc/rc3.d/**.

Al cambiar al runlevel X, *init* ejecuta el script **/etc/rc.d/rc**, que examina el directorio **/etc/rcX.d/** y ejecuta los enlaces que empiezan con S con el argumento *start* y los que empiezan por K con el argumento *stop*.

Se ejecutan primero los scripts con prefijo K y después S en el orden que indica su número. Esto garantiza que los servicios se detengan e inicien en el orden correcto según sus dependencias.

13.a

- ¿Qué es *systemd*?

Systemd es el sistema de inicio y gestor de procesos moderno predominante. Se ejecuta como primer proceso, PID 1, y es el responsable de inicializar el sistema, montar sistemas de archivos, gestionar sockets y temporizadores y supervisar la ejecución de todos los procesos.

13.b

- ¿A qué hace referencia el concepto de *unit* en *systemd*?

una unidad es un objeto de configuración que *systemd* sabe como administrar. Representa recursos o tareas del sistema como servicios, puntos de montaje, targets, etc.

Cada unidad se define mediante un archivo de configuración con una extensión que indica su tipo. Por ej. `.service`, `.mount`, `.target`.

Algunos comandos asociados son: `systemctl list-units` lista todas las unidades activas. `systemctl status <unidad>` informa el estado de una unidad y `systemctl start/stop/restart <unidad>`, controla una unidad.

13.c

- ¿Para qué sirve el comando *systemctl* en *systemd*?

systemctl es el comando principal para interactuar con *systemd*. Sirve para controlar y supervisar unidades del sistema.

Funciones principales:

1. gestión de servicios: `systemctl start ssh`; `systemctl stop ssh`; `systemctl reload ssh`
2. habilitar/deshabilitar servicios: `systemctl enable ssh`
3. consultar estado y logs: `systemctl status ssh`
4. listar unidades: `systemctl list-units`; `systemctl list-sockets`
5. gestionar *targets*: `systemctl get-default`; `systemctl set-default graphical.target`
6. apagar, reiniciar o suspender: `systemctl poweroff`; `systemctl reboot`; `systemctl suspend`
7. recarga su configuración: `sudo systemctl daemon-reload`

13.d

- ¿A qué hace referencia el concepto de *target* en systemd?

En systemd un *target* es un conjunto de unidades que define un estado o modo de operación del sistema. Es una unidad, **.target** que agrupa a otras, servicios, sockets, mounts, para alcanzar un estado deseado. Un *target* permite activar o desactivar un conjunto coherente de servicios de una vez.

Ejemplos comunes:

target	runlevel	desc.
poweroff.target	0	
rescue.target	1/S	
multi-user.target	3	
graphical.target	5	
reboot.target	6	

13.e

- Ejecutar el comando `ps tree`. ¿Qué es lo que se puede observar a partir de la ejecución de este comando?

Con **ps tree** se observa la jerarquía de procesos del sistema en forma de árbol. En distribuciones modernas, la raíz es systemd (PID 1), y de él cuelgan los demonios y servicios junto con sus procesos hijos. También se pueden ver las terminales activas y los procesos de usuario. La opción `-p` permite mostrar los PID.